

SQL Server & SSMS

8 Core Entities & Rules

INSTANT

Database Design Module • Session 23

ERD, MAPPING & NORMALIZATION

Instant Academy Business Case · 3NF Applied · SQL Server for Data Analysts ·
Database Fundamentals & DDL

8 Parts**Database Design**

Intro to 3NF Normalization

44 Slides**Interactive Deck**

Business Case & ERD Mapping

8 Tables**Instant Academy Schema**

3NF Relational Database

ERD Blueprint & Mapping

UNF to 3NF Normalization



01 • WHAT IS A DATABASE?

What is a Database?

Understanding why modern enterprises need structured relational data systems

— WHY COMPANIES NEED DATABASES

Why Do Companies Need Databases?

Imagine Instant Academy has 2,000 students, 50 courses, daily payments, and sales managing 500 leads/month. Can you track this in single Excel file?



Scenario: Imagine Instant Academy has 2,000 students, 50 courses, daily payments, and a sales team managing 500 leads per month. Could you track all of this reliably in a single Excel file?

× EXCEL PROBLEMS

- Duplicate student records across multiple files
- No way to link enrollments to payments automatically
- Accidental deletion destroys months of data
- Only one person can edit at a time

✓ DATABASE SOLUTIONS

- One central Students table — no duplicates
- Foreign keys link Enrollments to Payments
- Transactions & backups protect all data
- Hundreds of users query simultaneously

⚡ Interactive Scale Tester: Excel vs SQL Server

Drag sliders to test load

Rows: 50,000

Concurrent Users: 25

Excel Status: ⚠ Crash / Corrupted Lock

SQL Server Status: 🚀 0.02s Query Response Time

FOUR CORE PILLARS OF A DATABASE

What Exactly is a Database?

A database is an organized collection of structured data stored electronically, designed to be accessed, managed, queried, and updated simultaneously:

"A database is an organized collection of structured data stored electronically, designed to be easily accessed, managed, queried, and updated by multiple users simultaneously."



Structured

Data is organized into tables with defined columns and data types.



Controlled

Access permissions and rules enforce who can read or change data.



Relational

Tables can be linked together using shared primary/foreign keys.



Queryable

SQL lets you retrieve exactly the data you need in seconds.

— EXCEL VS SQL SERVER DATABASE

Database vs. Spreadsheet — A Comparison

As a data analyst, you will use BOTH — Excel for reports, SQL Server for querying large production databases.

Feature	Excel / Spreadsheet	SQL Server Database
Data Volume	Thousands of rows (limited)	Millions to billions of rows
Multi-user	One editor at a time	Hundreds of concurrent users
Data Integrity	Easy to break with typos	Enforced by constraints & data types
Relationships	Manual VLOOKUP / XLOOKUP	Native JOINS across relational tables
Automation	Manual refresh	Scheduled queries & stored procedures
Ideal For	Personal analysis & quick reports	Production business information systems

 **Key Takeaway:** As a data analyst, you will use BOTH — Excel for reports & dashboards, SQL Server for querying production databases.



02 • TABLES, KEYS & RELATIONSHIPS

Tables, Keys & Relationships

The building blocks of relational databases

— ANATOMY OF A DATABASE TABLE

Anatomy of a Database Table

Table (Entity) · Column (Attribute) · Row/Record (Instance) · Data Type (INT, VARCHAR, DATE, DECIMAL)

TABLE (ENTITY)

Represents one entity type (e.g. Students). Like a spreadsheet tab but strictly structured.

COLUMN (ATTRIBUTE)

A named attribute with a fixed data type (e.g., StudentName VARCHAR).

ROW / RECORD (INSTANCE)

One specific instance of the entity — one student, one payment.

DATA TYPE

Defines allowed values: INT, VARCHAR, DATE, DECIMAL, BIT.

Table: Students

StudentID (PK)	StudentName	Email	EnrollDate
1001	Sara Ahmed	sara@email.com	2024-01-15
1002	Omar Hassan	omar@email.com	2024-02-03
1003	Nour Khalil	nour@email.com	2024-02-20

Data Types: INT (PK) | VARCHAR(100) | VARCHAR(150) | DATE

— PRIMARY KEYS (PK) VS FOREIGN KEYS (FK)

Primary Keys and Foreign Keys

PK uniquely identifies each row in a table. FK references PK of another table to create relationships (1 -> Many).

PRIMARY KEY (PK)

- Uniquely identifies each row in a table
- Cannot be NULL or duplicated
- Every table should have one PK
- Example: `StudentID` in `Students` table

FOREIGN KEY (FK)

- References the Primary Key of another table
- Creates a relationship between tables
- Enforces referential integrity
- Example: `StudentID` in `Enrollments` table

Students

 `StudentID INT`

1 =====> Many

Enrollments

 `StudentID INT (FK)`



03 • SQL SERVER & YOUR TOOLKIT

SQL Server & Your Toolkit

Setting up your professional data analyst environment

— MICROSOFT SQL SERVER VS SSMS

SQL Server and SSMS — Your Toolkit

Quick Setup: Install SQL Server Express → Install SSMS → Connect to local instance (localhost\SQLEXPRESS)



Microsoft SQL Server

- The database engine that stores and manages your data
- Industry-leading RDBMS used by 90%+ of enterprises
- Handles millions of transactions per second
- Free Express edition for learning — full features for analysts
- Used in banking, healthcare, retail, and education



SSMS (Studio)

- SQL Server Management Studio — the visual GUI interface
- Write and run SQL queries in the Query Editor
- Browse databases, tables, and schemas via Object Explorer
- View query execution plans to optimize performance
- Free download from Microsoft — install in 10 minutes

⚙️ **Quick Setup:** Install SQL Server Express → Install SSMS → Connect to local instance (localhost\SQLEXPRESS) → You're ready!



04 • INSTANT ACADEMY BUSINESS CASE

Business Case

Understanding the Instant Academy information system

8 CORE BUSINESS ENTITIES

Instant Academy — Business Case Overview

Instant Academy is a private training institute. The system manages 8 core entities: Students, Courses, Campaigns, SalesAgents, Leads, Enrollments, Payments, Attendance.



Students

Registered individuals studying at the academy



Courses

Professional courses with fees & durations



Campaigns

Marketing campaigns across multiple channels



SalesAgents

Agents who convert leads into enrolled students



Leads

Potential students from campaigns or referrals



Enrollments

Students registered in one or more courses



Payments

Full or installment payments per enrollment



Attendance

Per-session attendance status per enrollment

WHAT THE SYSTEM MUST TRACK

Business Rules — What the System Must Track

Key attributes and constraints for each of the 8 entities

Students

Full name, email, city, enroll date. Can enroll in multiple courses. Converted from leads.

Courses

Category (AI, Cloud, Programming), tuition fee (DECIMAL), duration in hours. Many students per course.

Campaigns

Name, channel, budget, start date. Managed by exactly one Sales Agent. Generates multiple leads.

SalesAgents

Full name, region, hire date, target. Manages multiple campaigns & assigned multiple leads.

Leads

Name, source, status (New→Contacted→Interested→Converted/Lo). Linked to Agent & Campaign.

Enrollments

Student + Course + Date + Status (Active, Completed, Cancelled). One student has many enrollments.

Payments

Amount, date, method (Cash/Visa/Transfer). Multiple payments per enrollment (installments).

Attendance

Linked to Enrollment + Session Date. Status: Present, Absent, Late. One record per student per session.



05 • ERD BASICS

ERD Basics

Attributes, Relationship, Cardinality & Participation

— ENTITY-RELATIONSHIP DIAGRAM

What is an ERD?

An ERD is a visual blueprint of a database showing entities, attributes, and relationships — drawn BEFORE any table is created in SQL Server.

An Entity-Relationship Diagram (ERD) is a visual blueprint of a database. It shows the entities (things the system tracks), the attributes (properties describing each entity), and the relationships that connect entities to one another — before any table is ever created in SQL Server.



Entities

The "things" the business cares about — Students, Courses, Payments.



Attributes

The properties describing an entity — StudentName, Email, Price.



Relationships

How entities connect — a Student enrolls in a Course.

In short: the ERD is drawn first, on paper or in a tool — the SQL tables come second, mapped directly from it.

6 REASONS TO DRAW AN ERD

ERD Usages — Why We Draw One

Why database designers always draw an ERD before writing SQL code:



Design blueprint

Plan database structure before writing a single CREATE TABLE.



Team communication

Gives developers, analysts, and business stakeholders one shared picture.



Normalization basis

Reveals keys and dependencies needed to reach 1NF, 2NF, 3NF.



Documentation

Records entities, attributes, and business rules for future maintenance.



Schema mapping

Maps directly to relational tables — entities to tables, keys to PK/FK.



Reverse engineering

Used to document and understand an existing production database.

— STRONG (REGULAR) ENTITY

Strong (Regular) Entity

A strong entity has its own key attribute that uniquely identifies every record. Notation: single-lined rectangle.

Examples: Students, Courses, SalesAgents.

STRONG ENTITY

ERD Notation Standard

A strong entity has its own key attribute that uniquely identifies every record.
Notation: single-lined rectangle. Examples: Students, Courses, SalesAgents.

Notation: Single-lined rectangle [Entity]

Instant Academy Examples:

- Students — identified by StudentID
- Courses — identified by CourseID
- SalesAgents — identified by AgentID

Customer

Official ERD Standard Shape

— WEAK ENTITY

Weak Entity

A weak entity depends on a related strong (owner) entity's key + partial key to be identified. Notation: double-lined rectangle. Examples: PaymentInstallments, CourseSessions.

WEAK ENTITY

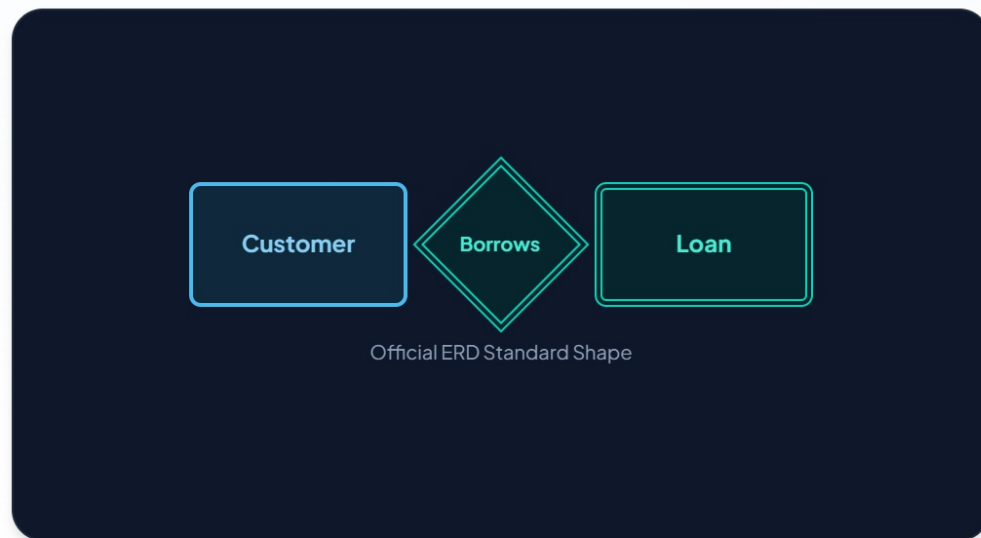
ERD Notation Standard

A weak entity depends on a related strong (owner) entity's key + partial key to be identified. Notation: double-lined rectangle. Examples: PaymentInstallments, CourseSessions.

Notation: Double-lined rectangle `[[Entity]]`, linked by double-lined diamond

Instant Academy Examples:

- PaymentInstallments — identified only with parent Payment
- CourseSessions — depends on parent Course
- Dependents — relative to Employee



— SIMPLE (ATOMIC) ATTRIBUTE

Simple (Atomic) Attribute

Holds a single, indivisible value. Notation: plain oval. Examples: Students.City, Courses.Price, Enrollments.EnrollStatus.

SIMPLE ATTRIBUTE

ERD Notation Standard

Holds a single, indivisible value. Notation: plain oval. Examples: Students.City, Courses.Price, Enrollments.EnrollStatus.

Notation: Plain single-line oval (Attribute)

Instant Academy Examples:

- Students.City — one indivisible value
- Courses.Price — single decimal amount
- Enrollments.EnrollStatus — single status



Official ERD Standard Shape

— KEY ATTRIBUTE

Key Attribute

Uniquely identifies every instance of an entity. Cannot be NULL or duplicate. Notation: oval with underlined text.

Examples: StudentID, CourseID, AgentID.

KEY ATTRIBUTE

ERD Notation Standard

Uniquely identifies every instance of an entity. Cannot be NULL or duplicate.
Notation: oval with underlined text. Examples: StudentID, CourseID, AgentID.

Notation: Oval with underlined text (<u>Roll_No</u>)

Instant Academy Examples:

- Students.StudentID – uniquely identifies student
- Courses.CourseID – primary key of Courses
- SalesAgents.AgentID – primary key of SalesAgents



Official ERD Standard Shape

COMPOSITE ATTRIBUTE

Composite Attribute

Can be broken down into smaller sub-attributes. Notation: oval with smaller ovals branching. Examples: StudentName -> FirstName + LastName; Address -> Street + City.

COMPOSITE ATTRIBUTE

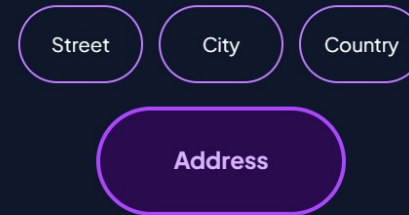
ERD Notation Standard

Can be broken down into smaller sub-attributes. Notation: oval with smaller ovals branching. Examples: StudentName -> FirstName + LastName; Address -> Street + City.

Notation: Oval with smaller sub-ovals branching from it

Instant Academy Examples:

- StudentName → FirstName + LastName
- Address → Street + City + PostalCode
- ContactInfo → Phone + Email



Official ERD Standard Shape

MULTI-VALUED ATTRIBUTE

Multi-Valued Attribute

Can hold more than one value for the same entity instance. Notation: double-lined oval. Examples: PhoneNumbers, Course Prerequisites.

MULTI-VALUED

ERD Notation Standard

Can hold more than one value for the same entity instance. Notation: double-lined oval. Examples: PhoneNumbers, Course Prerequisites.

Notation: Double-lined (double) oval ((Phone_No))

Instant Academy Examples:

- Student may have several PhoneNumbers
- Course may list multiple Prerequisites
- Campaign may run across several Channels



Official ERD Standard Shape

— DERIVED ATTRIBUTE

Derived Attribute

Calculated on the fly from other attributes rather than stored directly. Notation: dashed-outline oval. Examples: Age (from DateOfBirth), TotalPaid.

DERIVED ATTRIBUTE

ERD Notation Standard

Calculated on the fly from other attributes rather than stored directly. Notation: dashed-outline oval. Examples: Age (from DateOfBirth), TotalPaid.

Notation: Dashed-outline oval (:Age:)

Instant Academy Examples:

- Age → derived from DateOfBirth
- EnrollmentDuration → derived from EnrollDate & EndDate
- TotalPaid → derived by summing Payments



Official ERD Standard Shape

— ALL 5 ATTRIBUTE TYPES ON STUDENT ENTITY

Types of Attributes in ERD

Visual overview: Key (StudentID), Composite (Name -> First+Last), Simple (Address), Derived (Age), Multi-Valued (Courses).

Key (StudentID)

Simple (City)

Composite (Name)

Multi-Valued (Phones)

Derived (Age)

1. Key Attribute (oval with underlined text):

StudentID – uniquely identifies every student record. Cannot be NULL or duplicated.

— DEGREE 1 · UNARY (RECURSIVE)

Unary (Recursive) Relationship

Connects instances of the SAME entity type to one another. Examples: SalesAgent supervises SalesAgent; Course is prerequisite of Course.

DEGREE 1 (UNARY / RECURSIVE)

Connects instances of the SAME entity type to one another. Examples: SalesAgent supervises SalesAgent; Course is prerequisite of Course.

Instant Academy Examples:

- SalesAgent supervises another SalesAgent
- Course is a prerequisite of another Course
- Employee reports to another Employee



Relationship Diagram Degree

— DEGREE 2 · BINARY RELATIONSHIP

Binary Relationship

Connects instances of TWO different entity types. Most common degree in ERDs (1:1, 1:N, N:M). Examples: Student enrolls in Course.

DEGREE 2 (BINARY)

Connects instances of TWO different entity types. Most common degree in ERDs (1:1, 1:N, N:M). Examples: Student enrolls in Course.

Instant Academy Examples:

- Student enrolls in a Course (N:M)
- SalesAgent manages a Campaign (1:N)
- Payment belongs to one Enrollment (1:N)



Relationship Diagram Degree

— DEGREE 3 · TERNARY RELATIONSHIP

Ternary Relationship

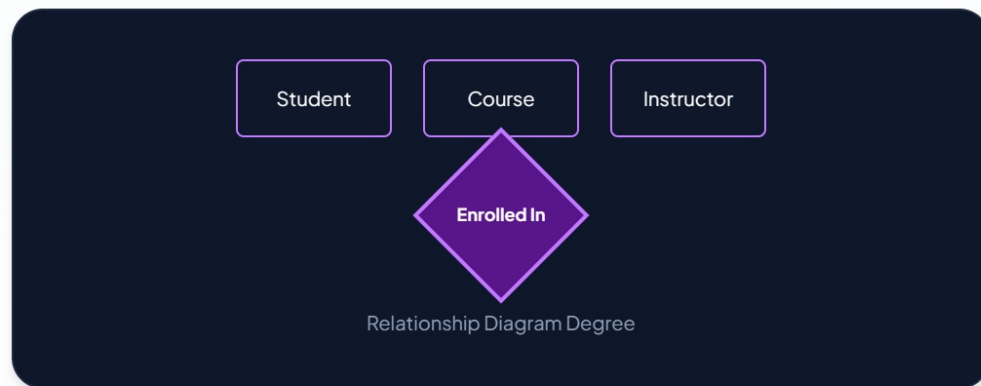
Links instances of THREE entity types in a single relationship. Example: Student books a Course in a specific Session with an Instructor.

DEGREE 3 (TERNARY)

Links instances of THREE entity types in a single relationship. Example: Student books a Course in a specific Session with an Instructor.

Instant Academy Examples:

- SalesAgent assigns a Lead through a Campaign in a given Month
- Student books a Course in a specific Session with a specific Instructor



— 1:1, 1:N, AND N:M CARDINALITY

Cardinality — How Many Instances Relate?

1:1 (One-to-One), 1:N (One-to-Many), N:M (Many-to-Many requires junction table in SQL)

Cardinality defines the maximum number of times one entity instance can be associated with instances of another entity.

1:1 (One-to-One)

🎓 One SalesAgent has one LoginAccount

One instance of A is related to exactly one instance of B.

1:N (One-to-Many)

🎓 One SalesAgent manages many Campaigns

One instance of A is related to many instances of B, but B relates to only one A.

N:M (Many-to-Many)

🎓 Students enroll in many Courses; Courses have many Students

Many instances of A relate to many instances of B. Requires a junction table in SQL.

TOTAL (MANDATORY) VS PARTIAL (OPTIONAL)

Participation Constraints — Is it Mandatory?

Total Participation (Double line ==) vs Partial Participation (Single line —) + Instant Academy Summary Table

Total Participation (Mandatory)

Notation: Double line ==

Every entity instance MUST participate in the relationship. No exceptions allowed.

Example: Every Enrollment MUST have a Student. An enrollment without a student makes no sense.

Partial Participation (Optional)

Notation: Single line —

Some entity instances may NOT participate in the relationship — it is optional.

Example: A Student may or may not have Payments yet (e.g. scholarship, not yet invoiced).

Entity	Target	Participation	Rule Summary
Enrollment	Student	Total (==)	Every enrollment must have a student
Enrollment	Course	Total (==)	Every enrollment must reference a course
Payment	Enrollment	Total (==)	Every payment must belong to an enrollment
Attendance	Enrollment	Total (==)	Every attendance record requires an enrollment
Student	Enrollment	Partial (—)	A student may exist before any enrollment
Lead	Campaign	Partial (—)	A lead may arrive directly (walk-in) with no campaign



06 • INSTANT ACADEMY ER DIAGRAM

ER Diagram

Instant Academy — Full ERD with 8 Entities & 7 Relationships

— FULL 8-ENTITY ERD MAP

Instant Academy — Entity Relationship Diagram

Complete blueprint showing Students, Courses, Enrollments, Payments, Attendance, Leads, SalesAgents, Campaigns.

 Instant Academy ERD Blueprint: 8 Core Entities + 7 Diamond Relationships

[Click an entity to inspect](#)

Students

Courses

SalesAgents

Campaigns

Leads

Enrollments

Payments

Attendance

 Entity Details: Students

Identified by StudentID. Attributes: StudentName, Email, City, EnrollDate, Status, NationalID



07 • ERD TO RELATIONAL MAPPING

ERD → Relational Mapping

Translating the visual ER Diagram directly into SQL Relational Tables

— 7 STANDARD MAPPING RULES

Mapping Rules — ERD to Relational Tables

Rule 1: Strong Entity -> Table | Rule 2: Weak Entity -> Table+FK | Rule 3: 1:1 -> Merge FK | Rule 4: 1:N -> FK on N-side | Rule 5: M:N -> Junction Table | Rule 6: Multi-Valued -> New Table | Rule 7: Composite -> Flat Columns



Rule 1: Strong Entity

Becomes a table. Attributes become columns. Key attribute becomes PRIMARY KEY.



Rule 2: Weak Entity

Becomes a table. PK = own partial key + FK of owner entity (composite PK).



Rule 3: 1:1 Relationship

Place FK in either entity (prefer total participation). Add UNIQUE on FK.



Rule 4: 1:N Relationship

The FK goes in the entity on the MANY side, referencing PK of ONE side.



Rule 5: M:N Relationship

Create a new junction table with FKs to both entities. PK = composite of both FKs.



Rule 6 & 7: Attributes

Multi-valued becomes new table. Composite breaks into flat atomic columns.

— ALL 8 ENTITIES MAPPED TO SQL TABLES

Mapping Result — Instant Academy Relational Schema

All 8 entities mapped to SQL tables with Primary Keys (PKs), Foreign Keys (FKs), and column data types.

Students

- **StudentID** INT (PK)
- StudentName NVARCHAR
- Email VARCHAR UNIQUE
- City NVARCHAR
- EnrollDate DATE

Courses

- **CourseID** INT (PK)
- CourseName NVARCHAR
- Category VARCHAR
- Price DECIMAL
- DurationHrs INT

SalesAgents

- **AgentID** INT (PK)
- AgentName NVARCHAR
- Email, Phone, Region
- MonthlyTarget DECIMAL
- IsActive BIT

Campaigns

- **CampaignID** INT (PK)
- **AgentID** (FK)
- CampaignName NVARCHAR
- Channel, Budget
- StartDate, EndDate

Leads

- **LeadID** INT (PK)
- **AgentID** (FK)
- **CampaignID** (FK)
- LeadName, Source, Status

Enrollments

- **EnrollmentID** INT (PK)
- **StudentID** (FK)
- **CourseID** (FK)
- EnrollDate, EnrollStatus

Payments

- **PaymentID** INT (PK)
- **EnrollmentID** (FK)
- Amount DECIMAL
- PaymentDate, Method

Attendance

- **AttendanceID** INT (PK)
- **EnrollmentID** (FK)
- SessionDate, AttendStatus
- Notes NVARCHAR



08 • NORMALIZATION UP TO 3NF

Normalization

Why, What, and How — structuring relational tables up to 3NF

— ANOMALIES VS BENEFITS OF NORMALIZATION

What is Normalization — and Why Do We Use It?

Eliminating Insert, Update, Delete anomalies and Redundancy vs Benefits: No Redundancy, Data Integrity, Smaller Storage.

Normalization is a systematic process of structuring a relational database to reduce data redundancy, eliminate anomalies, and ensure data integrity — by organizing attributes into well-structured tables following Normal Forms (NF).

⚠ PROBLEMS WITHOUT NORMALIZATION

- Insert Anomaly:** Cannot add payment without enrollment
- Update Anomaly:** Changing student city requires updating 100 rows
- Delete Anomaly:** Deleting last enrollment loses student record
- Data Redundancy:** 'SQL Server' stored 500 times
- Inconsistency:** 'Cairo' in one row, 'Al-Qahira' in another

✅ BENEFITS WITH NORMALIZATION

- No Redundancy:** Each fact stored exactly once
- Easy Updates:** Change student city in ONE row
- Data Integrity:** FK constraints enforce valid links
- Smaller Storage:** Less disk space & faster queries
- Easier Queries:** Structured tables are clean to JOIN

— 1NF, 2NF, AND 3NF DEFINITIONS

Normal Forms — The Stages of Normalization

1NF: Atomic values (no repeating groups) | 2NF: 1NF + no partial dependencies on composite PK | 3NF: 2NF + no transitive dependencies

1NF — First Normal Form

Prerequisite: No repeating groups or arrays. Atomic values only.

X Violation: Storing 'Cash, Visa' in one PaymentMethod cell

✓ Fix: One row per payment — one Method per row

2NF — Second Normal Form

Prerequisite: Must be in 1NF + No partial dependencies on composite PK.

X Violation: In (StudentID, CourseID) → CourseName depends only on CourseID

✓ Fix: Move CourseName to the Courses table

3NF — Third Normal Form

Prerequisite: Must be in 2NF + No transitive dependencies.

X Violation: EnrollmentID → AgentID → AgentName (AgentName depends on AgentID)

✓ Fix: Move Agent data to SalesAgents table; store AgentID as FK

RAW UNNORMALIZED TABLE: ACADEMYDATA

Step 0 — Unnormalized Data (UNF)

Everything crammed into one flat table — Redundancy (Sara Ahmed x3), Update Anomaly (change city in 3 rows), Delete Anomaly.

🔔 Imagine all academy data in ONE single flat spreadsheet. This is our starting point BEFORE normalization.

StudentID	StudentName	City	CourseName	Category	Price	AgentName	Region	CampaignName	Channel	Amount	Method	SessionDate	Status
1001	Sara Ahmed	Cairo	SQL Server	Data	4500	Ahmed Ali	Cairo	Facebook Q1	Facebook	4500	Visa	2024-03-01	Present
1001	Sara Ahmed	Cairo	SQL Server	Data	4500	Ahmed Ali	Cairo	Facebook Q1	Facebook	1500	Cash	2024-03-08	Absent
1001	Sara Ahmed	Cairo	Power BI	Data	3200	Ahmed Ali	Cairo	Facebook Q1	Facebook	3200	Transfer	2024-03-15	Present
1002	Omar Hassan	Alex	Python	Prog	5000	Mona Samir	Alex	Google Q1	Google	2500	Visa	2024-03-01	Late
1002	Omar Hassan	Alex	Python	Prog	5000	Mona Samir	Alex	Google Q1	Google	2500	Cash	2024-03-08	Present

Redundancy: Sara Ahmed x 3 rows

Update Anomaly: Change City in 3 rows

Delete Anomaly: Delete payments = lose student

— ATOMIC VALUES + NO REPEATING GROUPS

Step 1 — First Normal Form (1NF)

Violation: Storing 'SQL Server, Power BI' in one cell -> Fix: Atomic rows (one value per cell).

Rule: Every column must hold ONE atomic value. No comma-separated lists. Every row must be unique.

✗ Before 1NF – Violation Example

EnrollID	Name	Courses (list!)	Methods (list!)
E001	Sara Ahmed	SQL Server, Power BI	Visa, Cash
E002	Omar Hassan	Python	Visa, Cash

Storing comma lists violates atomicity – cannot filter or join properly.

✓ After 1NF – Atomic Rows

EnrollID	Name	Course	Method
E001	Sara Ahmed	SQL Server	Visa
E001	Sara Ahmed	SQL Server	Cash
E001	Sara Ahmed	Power BI	Visa
E002	Omar Hassan	Python	Visa

✓ 1NF achieved: atomic values, no lists | ⚠ Still has issue: repeated StudentName + CourseName → need 2NF

— ELIMINATE PARTIAL DEPENDENCIES

Step 2 — Second Normal Form (2NF)

Violation: In (StudentID, CourseID), StudentName depends only on StudentID -> Fix: Split into Students, Courses, Enrollments.

Rule: Must be in 1NF + every non-key column must depend on the WHOLE primary key (applies when PK is composite).

✗ Partial Dependency Violation

Composite PK: (StudentID + CourseID)

StudentName depends on: StudentID only ✗

City depends on: StudentID only ✗

CourseName depends on: CourseID only ✗

Price depends on: CourseID only ✗

EnrollDate depends on: StudentID + CourseID ✓

✓ After 2NF – Tables Split

Students (PK: StudentID): StudentName, Email, City, EnrollDate

Courses (PK: CourseID): CourseName, Category, Price, DurationHrs

Enrollments (PK: StudentID + CourseID): EnrollDate, EnrollStatus

💡 Academy fix: Enrollments now only stores EnrollmentID, StudentID (FK), CourseID (FK), and enrollment-specific data.

— ELIMINATE TRANSITIVE DEPENDENCIES

Step 3 — Third Normal Form (3NF)

Violation: LeadID → AgentID → AgentName (AgentName depends on AgentID) → Fix: Separate Leads, SalesAgents, Campaigns.

Rule: Must be in 2NF + no non-key column should depend on another non-key column. All facts must depend on the PK — the whole PK — and nothing but the PK.

✗ Transitive Dependency in Leads Table

LeadID (PK) → AgentID (non-key) → AgentName (transitive!)

AgentName does NOT depend on LeadID — it depends on AgentID (another non-key)!

✓ After 3NF — Tables Separated

Leads (PK: LeadID): FK: AgentID, CampaignID | Own: LeadName, Source

SalesAgents (PK: AgentID): Own: AgentName, Region, MonthlyTarget

Campaigns (PK: CampaignID): FK: AgentID | Own: CampaignName, Budget

🎉 3NF Result: Each table describes exactly ONE thing. No attribute depends on anything other than the primary key.

— ALL 8 TABLES + SAMPLE DATA

Final 3NF Schema — Instant Academy

Clean 8-table relational schema: Students, Courses, SalesAgents, Campaigns, Leads, Enrollments, Payments, Attendance.

 Final 3NF Schema & Production Sample Data: All 8 Normalized Tables

Click tab to switch sample table data

Students

Courses

SalesAgents

Enrollments

StudentID	StudentName	Email	City	EnrollDate	Status
1001	Sara Ahmed	sara@mail.com	Cairo	2024-01-10	Active
1002	Omar Hassan	omar@mail.com	Alex	2024-02-03	Active
1003	Nour Khalil	nour@mail.com	Cairo	2024-03-01	Active

— SESSION 23 RECAP

Session Summary — What You Learned

Recap of Business Case, ERD Concepts, ER Diagram, Mapping Rules, and 3NF Normalization:



1. Business Case

8 entities: Students, Courses, Campaigns, SalesAgents, Leads, Enrollments, Payments, Attendance.



2. ERD Concepts

5 attribute types (Simple, Key, Composite, Multi-valued, Derived) & Cardinalities (1:1, 1:N, N:M).



3. ER Diagram

8 entities drawn with proper notation, 7 relationships with diamond notation & cardinality labels.



4. Mapping Rules

7 standard rules applied to convert ERD into SQL tables with PKs and FKs.



5. Normalization 3NF

UNF -> 1NF (atomic) -> 2NF (no partial) -> 3NF (no transitive) = clean 8-table schema.

INSTANT

✓ SESSION 23 COMPLETED SUCCESSFULLY

THANK YOU

Thank you for your active participation!

🎉 **Session 23 Complete! You have mastered Database Fundamentals, ERD Diagramming & 3NF Normalization!**

↺ Restart Presentation

Open Session 24: SQL Server DDL Statements & Queries →